1. Project Overview

Field	Details
Project Name	student-records-app
Backend	Node.js (v20-alpine) — Express REST API
Database	PostgreSQL 16 (alpine)
Networking	Docker MacVLAN (student_macvlan)
Orchestration	Docker Compose
API IP Address	192.168.1.100:3000
DB IP Address	192.168.1.101:5432

2. Project Structure

The project was initialized with the following directory layout:

```
student-records-app/
├── backend/
│   ├── Dockerfile
│   ├── index.js
│   ├── package.json
│   └── .dockerignore
├── database/
│   ├── Dockerfile
│   └── init.sql
├── docker-compose.yml
├── network_setup.sh
└── .env
```

2.1 Setup Commands

The project directory and all required files were created using the following shell commands:

```
mkdir student-records-app && cd student-records-app
mkdir backend database
touch backend/index.js backend/package.json backend/Dockerfile
backend/.dockerignore
touch database/Dockerfile database/init.sql
touch docker-compose.yml
touch network_setup.sh
touch .env
```

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops$ mkdir student-records-app
aniket@Aniket:/mnt/d/ccvt/sem 6/devops$ cd student-records-app
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ mkdir backend
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ mkdir database
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch backend/index.js
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch backend/package.json
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch backend/Dockerfile
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch backend/.dockerignore
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch database/Dockerfile
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch database/init.sql
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch docker-compose.yml
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch network_setup.sh
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ touch .env
```

3. Component Descriptions

3.1 Backend (Node.js)

File: backend/index.js

The backend is a Node.js Express application that provides a REST API for student CRUD operations. It connects to the PostgreSQL database using the pg library and initializes the students table on startup.

Endpoint	Description
GET /health	Health check — returns {status: 'ok'}
GET /students	Retrieve all student records
POST /students	Add a new student record

3.2 Backend Dockerfile

The backend uses a multi-stage build for a lean production image:

- Stage 1 (builder): Uses node:20-alpine, installs npm dependencies
- Stage 2 (production): Copies built app, creates a non-root user (appuser) for security
- Exposes port 3000

3.3 Database (PostgreSQL)

File: database/init.sql

A custom PostgreSQL image is built from postgres:16-alpine. The init.sql script is placed in /docker-entrypoint-initdb.d/ so it runs automatically on first container startup. It creates the students table with the schema: id (serial PK), name, roll_number, department, year.

3.4 Docker Compose

File: docker-compose.yml

The Compose file defines two services (backend, database) and one named volume (student_pgdata). Both services are attached to the custom student_macvlan network with static IPs assigned:

- studentapi container → 192.168.1.100
- studentdb container → 192.168.1.101

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker inspect studentapi | grep IPAddress
                    "IPAddress": "192.168.1.100",
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ |
```

3.5 Environment Configuration

File: .env

Sensitive configuration (database credentials, connection strings) is stored in .env and referenced by Docker Compose. This keeps secrets out of source code and Dockerfiles.

4. Docker MacVLAN Network Setup

The most significant aspect of this assignment is the use of a MacVLAN Docker network, which allows containers to appear as physical devices on the host network with their own MAC addresses.

4.1 Creating the MacVLAN Network

File: network_setup.sh

The script creates a Docker MacVLAN network using the host's eth0 interface:

```
docker network create \
  --driver macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  --ip-range=192.168.1.96/28 \
  -o parent=eth0 \
  student_macvlan
```

4.2 Host-Side MacVLAN Interface

To allow the host machine to communicate with containers on the MacVLAN network, a macvlan0 sub-interface is created on the host:

```
sudo ip link add macvlan0 link eth0 type macvlan mode bridge
sudo ip addr add 192.168.1.99/32 dev macvlan0
sudo ip link set macvlan0 up
sudo ip route add 192.168.1.96/28 dev macvlan0
```

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker inspect studentapi | grep IPAddress
                    "IPAddress": "192.168.1.100",
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker exec -it studentapi sh
/app $ wget -qO- http://localhost:3000/health
{"status":"ok"}/app $ exit
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ sudo ip link add macvlan0 link eth0 type macvlan mode bridge
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ sudo ip addr add 192.168.1.99/32 dev macvlan0
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ sudo ip link set macvlan0 up
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ sudo ip route add 192.168.1.96/28 dev macvlan0
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ curl http://192.168.1.100:3000/health
{"status":"ok"}aniket@Aniket:/mnt/d/ccvt/sem 6/devops/studencurl -X POST http://192.168.1.100:3000/students \3000/students \
-H "Content-Type: application/json" \
-d '{"name":"Aniket","roll_number":"CS01","department":"CSE","year":3}'
{"id":1,"name":"Aniket","roll_number":"CS01","department":"CSE","year":3}aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ curl http://192.168.1.1
00:3000/students
ls: cannot access 'http://192.168.1.100:3000/students': No such file or directory
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ curl http://192.168.1.100:3000/students
[{"id":1,"name":"Aniket","roll_number":"CS01","department":"CSE","year":3}]aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker ps
```

4.3 Network Verification

After setup, the network was verified using docker network inspect student_macvlan, which showed:

- Subnet: 192.168.1.0/24, Gateway: 192.168.1.1
- studentapi assigned MAC 62:aa:32:ba:95:58 at IPv4 192.168.1.100/24
- studentdb assigned MAC 42:e8:53:00:5a:6d at IPv4 192.168.1.101/24


```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker network inspect student_macvlan
[
    {
        "Name": "student_macvlan",
        "Id": "205dea74d98cc64d34e8b8ac721e70c6db92b0a573d6698b26ae605d70739959",
        "Created": "2026-03-15T23:30:01.983075881+05:30",
        "Scope": "local",
        "Driver": "macvlan",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "192.168.1.0/24",
                    "Gateway": "192.168.1.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "parent": "eth0"
        },
        "Labels": {},
        "Containers": {
            "230196d119d92fcdf705a57cffc39a40a5b07990f593482d9cb2243a5e16b099": {
                "Name": "studentapi",
                "EndpointID": "1d88206958b0d20df795432128ff137f340a68ad258a15fe9a710fe0ecf350b2",
                "MacAddress": "62:aa:32:ba:95:58",
                "IPv4Address": "192.168.1.100/24",
                "IPv6Address": ""
            },
            "98c4f268b2c1964f0207ec5686df316ee8fadc5efac60d85fa6a84d9a0b3eb67": {
                "Name": "studentdb",
                "EndpointID": "423f8ba367c5a3baba45784cbc194f0c1741aabe3a2cc4a0c6047dfbfd7b5cfb",
                "MacAddress": "42:e8:53:00:5a:6d",
```


5. Build & Deployment

5.1 Building and Starting Containers

```
docker compose up --build -d
```

The build process completed successfully in ~11 seconds:

- student-records-app-backend image built (Node.js multi-stage)
- student-records-app-database image built (PostgreSQL with init.sql)
- Volume student-records-app_student_pgdata created
- Containers studentapi and studentdb started

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker compose up --build -d
WARN[0000] /mnt/d/ccvt/sem 6/devops/student-records-app/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avo
id potential confusion
[+] Building 10.9s (22/22) FINISHED
 => [internal] load local bake definitions                                0.0s
 => => reading from stdin 1.09kB                                          0.0s
 => [database internal] load build definition from Dockerfile             0.1s
 => => transferring dockerfile: 197B                                      0.0s
 => [backend internal] load build definition from Dockerfile              0.1s
 => => transferring dockerfile: 311B                                      0.0s
 => [database internal] load metadata for docker.io/library/postgres:     0.1s
 => [backend internal] load metadata for docker.io/library/node:20-al     0.1s
 => [database internal] load .dockerignore                                0.1s
 => => transferring context: 2B                                           0.0s
 => [backend internal] load .dockerignore                                 0.2s
 => => transferring context: 63B                                          0.1s
 => [database internal] load build context                                0.2s
 => => transferring context: 182B                                         0.1s
 => [database 1/2] FROM docker.io/library/postgres:16-alpine@sha256:2     1.5s
 => => resolve docker.io/library/postgres:16-alpine@sha256:20edbde774     0.1s
 => [backend internal] load build context                                 0.3s
 => => transferring context: 1.90kB                                       0.0s
 => [backend builder 1/5] FROM docker.io/library/node:20-alpine@sha25     0.6s
 => => resolve docker.io/library/node:20-alpine@sha256:b88333c42c23fb     0.1s
 => [backend builder 2/5] WORKDIR /app                                    0.3s
 => [backend builder 3/5] COPY package*.json ./                           0.3s
 => [backend stage-1 3/4] RUN addgroup -S appgroup && adduser -S appu     1.6s
 => [backend builder 4/5] RUN npm install                                 6.3s
 => [database 2/2] COPY init.sql /docker-entrypoint-initdb.d/             0.4s
```

```
 => [backend stage-1 4/4] COPY --from=builder /app /app                   0.2s
 => [backend] exporting to image                                          1.2s
 => => exporting layers                                                   0.6s
 => => exporting manifest sha256:8f456362fd3101a2792ea27658bb82f1dbae     0.0s
 => => exporting config sha256:10fb95ea668ea9f23d69ce43e377d8e062f5ee     0.0s
 => => exporting attestation manifest sha256:12d5a903eef789ad54800caf     0.1s
 => => exporting manifest list sha256:8bc56e9b2bf13916dcb700d714d2ef2     0.0s
 => => naming to docker.io/library/student-records-app-backend:latest     0.0s
 => => unpacking to docker.io/library/student-records-app-backend:lat     0.4s
 => [backend] resolving provenance for metadata file                      0.0s
[+] up 5/5
 ✔Image student-records-app-backend         Built                        11.1s
 ✔Image student-records-app-database        Built                        11.1s
 ✔Volume student-records-app_student_pgdata Created                      0.0s
 ✔Container studentdb                       Created                      0.2s
 ✔Container studentapi                      Created                      0.1s
```

6. Verification & Testing

6.1 Health Check

```
docker exec -it studentapi sh
wget -qO- http://localhost:3000/health
# Output: {"status":"ok"}
```

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker exec -it studentapi sh
/app $ wget -qO- http://localhost:3000/health
{"status":"ok"}/app $ exit
```

```
curl http://192.168.1.100:3000/health
# Output: {"status":"ok"}
```

6.2 Creating Student Records

```
curl -X POST http://192.168.1.100:3000/students \
-H "Content-Type: application/json" \
-d '{"name":"Aniket","roll_number":"CS01","department":"CSE","year":3}'
# Output: {"id":1,"name":"Aniket","roll_number":"CS01","department":"CSE","year":3}
```

```
curl -X POST http://192.168.1.100:3000/students \
-H "Content-Type: application/json" \
-d '{"name":"Rahul","roll_number":"CS02","department":"CSE","year":3}'
```

6.3 Listing All Students

```
curl http://192.168.1.100:3000/students
[{"id":1,"name":"Aniket","roll_number":"CS01","department":"CSE","year":3},
 {"id":2,"name":"Rahul","roll_number":"CS02","department":"CSE","year":3}]
```

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ curl http://192.168.1.100:3000/students
[{"id":1,"name":"Aniket","roll_number":"CS01","department":"CSE","year":3}]aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ curl -X POST http://1
92.168.1.100:3000/students \                                    curl -X POST http://192.168.1.100:3000/students \
-H "Content-Type: application/json" \
-d '{"name":"Rahul","roll_number":"CS02","department":"CSE","year":3}'
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ curl http://192.168.1.100:3000/students
[{"id":1,"name":"Aniket","roll_number":"CS01","department":"CSE","year":3},{"id":2,"name":"Rahul","roll_number":"CS02","department":"CSE","year":3}]aniket@A
niket:/mnt/d/ccvt/sem 6/devops/student-records-app$
```

7. Persistent Storage

Data persistence is achieved using a named Docker volume. The volume student-records-app_student_pgdata is managed by Docker Compose and mounts to /var/lib/postgresql/data inside the studentdb container.

Property	Value
Volume Name	student-records-app_student_pgdata
Driver	local
Mount Point	/var/lib/docker/volumes/student-records-app_student_pgdata/_data
Created At	2026-03-15T23:40:33+05:30
Compose Project	student-records-app

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker volume inspect student-records-app_student_pgdata
[
    {
        "CreatedAt": "2026-03-15T23:40:33+05:30",
        "Driver": "local",
        "Labels": {
            "com.docker.compose.config-hash": "3cfcc7d38301edd8da4302911e2b605a51d03f823267cacf587461bfa709f637",
            "com.docker.compose.project": "student-records-app",
            "com.docker.compose.version": "5.0.2",
            "com.docker.compose.volume": "student_pgdata"
        },
        "Mountpoint": "/var/lib/docker/volumes/student-records-app_student_pgdata/_data",
        "Name": "student-records-app_student_pgdata",
        "Options": null,
        "Scope": "local"
    }
]
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$
```

8. Running Containers

The docker ps output showed the following active containers at the time of verification:

Name	Image	Status	Notes
studentapi	`student-records-app-backend`	Up	REST API — 192.168.1.100:3000
studentdb	`student-records-app-database`	Up	PostgreSQL — 192.168.1.101:5432

```
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ curl http://192.168.1.100:3000/students
[{"id":1,"name":"Aniket","roll_number":"CS01","department":"CSE","year":3}]aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$ docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS          PORTS     NAMES
230196d119d9   student-records-app-backend    "docker-entrypoint.s…"   9 minutes ago    Up 9 minutes              studentapi
98c4f268b2c1   student-records-app-database   "docker-entrypoint.s…"   9 minutes ago    Up 9 minutes              studentdb
96646fc3870e   nginx:latest                   "/docker-entrypoint.…"   48 minutes ago   Up 48 minutes   80/tcp    webapp.6.k09ht65uhkgitpzn2j96o53dq
d804c0c7020a   nginx:latest                   "/docker-entrypoint.…"   48 minutes ago   Up 48 minutes   80/tcp    webapp.4.6c64hdgtfxqw6fc9wortq3uro
1c27f81dfd04   nginx:latest                   "/docker-entrypoint.…"   48 minutes ago   Up 48 minutes   80/tcp    webapp.1.0pcqh05yq6o3uhujkzy9xsxvb
3e408e2a67eb   nginx:latest                   "/docker-entrypoint.…"   48 minutes ago   Up 48 minutes   80/tcp    webapp.5.mnpg9hwuaemr0ezvgzlazzv37
e655e9a31d74   nginx:latest                   "/docker-entrypoint.…"   48 minutes ago   Up 48 minutes   80/tcp    webapp.2.kkwyzxckm67f3o0wuso60dkyx
dd94029af6f1   nginx:latest                   "/docker-entrypoint.…"   48 minutes ago   Up 48 minutes   80/tcp    webapp.3.blx4n2tfr8tuwh7eheofzxdyj
aniket@Aniket:/mnt/d/ccvt/sem 6/devops/student-records-app$
```

10. Conclusion

The Student Records App was successfully deployed using Docker Compose with a MacVLAN network. Key achievements:

- Containerized a full Node.js + PostgreSQL stack with separate Dockerfiles for each service
- Configured a Docker MacVLAN network (student_macvlan) giving each container a unique IP on the host subnet
- Resolved Docker Hub auth expiry and macvlan host-routing issues during deployment
- Verified REST API endpoints (health check, GET /students, POST /students) via curl
- Confirmed data persistence using a named Docker volume (student_pgdata)

The assignment demonstrates practical experience with containerization, custom networking, multi-service orchestration, and Docker volume management — core DevOps competencies.